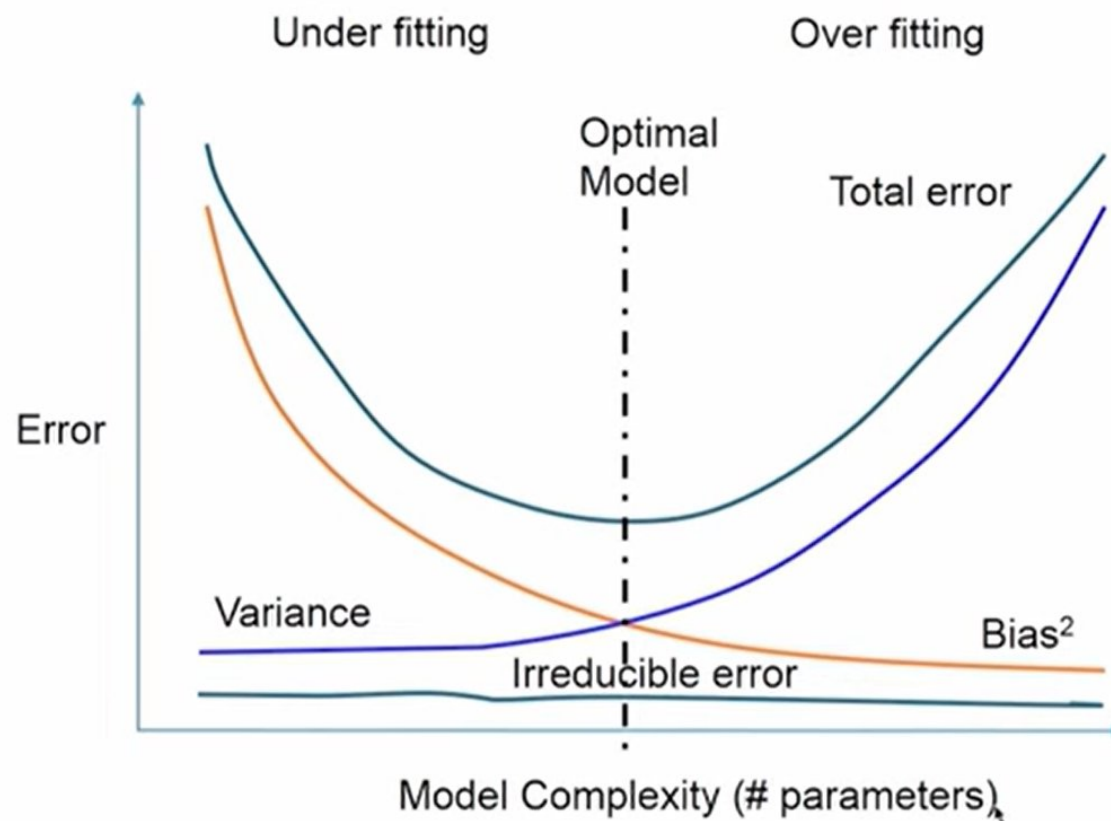


CROSS VALIDATION

Motivation

- How to select the optimal number of meta or hyper-parameters of a model?
 - Number of principal components in principal components analysis
 - Number of clusters in K-means clustering
 - Number of terms ' n ' in polynomial or nonlinear regression
$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots \beta_n x^n$$
(equivalent to multilinear regression by treating $x, x^2, \dots x^n$ as different variables)
- MSE of **training data set not useful as a measure**
 - MSE will decrease with increasing number of parameters (can be reduced to zero)
- Use cross validation on **a validation data set** to determine optimal number of parameters

Bias-Variance trade-off on test data set



Training and Validation data sets

- For large data sets divide data set into training data set ($\sim 70\%$ of the samples) and remaining validation/test data
 - Training set: $\{(\mathbf{x}_1, y_1); (\mathbf{x}_2, y_2); \dots; (\mathbf{x}_n, y_n)\}$
 - Test set: $(\mathbf{x}_{0,i}, y_{0,i}) : i = 1 \dots n_t$ observations

- Training error rate

$$MSE_{Training} = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \hat{\beta})^2$$

- Test error rates

$$MSE_{Test} = \frac{1}{n_t} \sum_{i=1}^n (y_{0,i} - \mathbf{x}_{0,i}^T \hat{\beta})^2$$

Validation Set Approach

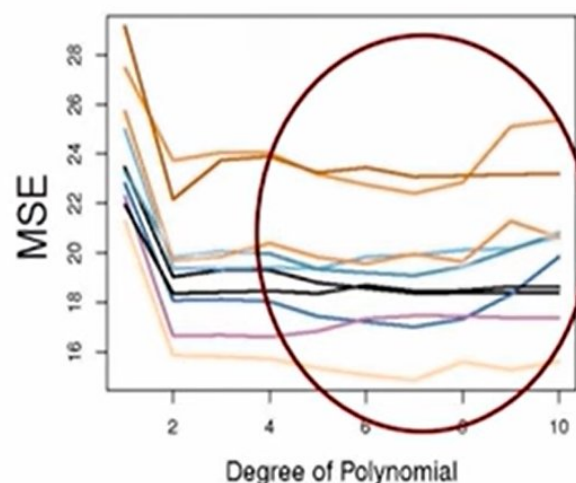
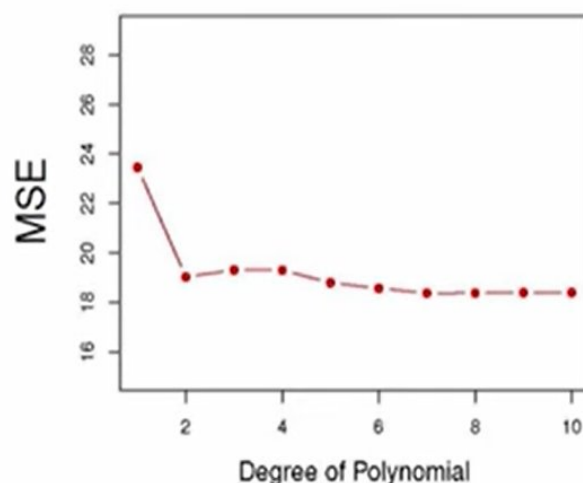
- Enough data: (1) Training set, (2) Validation set, and (3) Test set
- Not enough data: Generate validation sets from a training set
- Validation set approach: Divides (often randomly) the training set into two parts

	1 2 3 4	n
• A training set	1 2 3 4	n_t
• A validation set (or hold-out set)	1 2 3 4	n_v

- Use training set, to fit the model
- Use validation set, to predict validation set errors
Provides an estimate of test error rates

Validation Set Approach: Example

- Example: $\text{mileage} \sim \text{horsepower}^1$ (> 300 data points on horsepower of automobiles and mileage)
- Polynomial Model: $\text{mileage} \sim f(\text{horsepower})$



High variability in estimates of test error

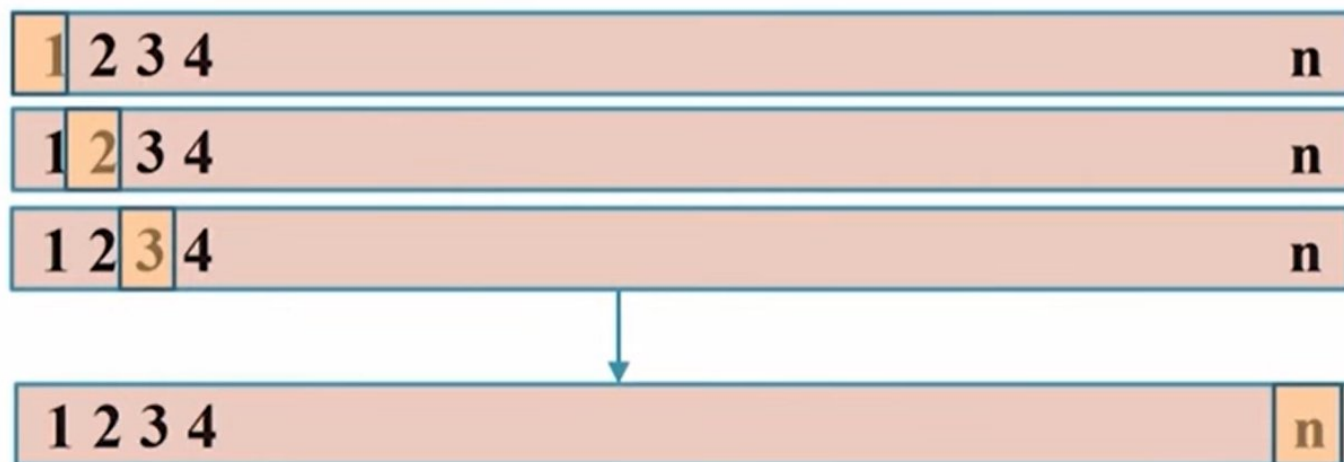
¹Tibshirani et al (2013)

Sampling for small data sets

- Validation of models by repeatedly drawing random samples from a training set
 - Validation set (random sampling)
 - K-fold cross validation
 - Bootstrap
- Objective: Predict the performance of model(s) on the validation/test sets (drawn from training data)
- Resampling methods useful for data scarce situations

Leave-one-out-cross-validation (LOOCV)

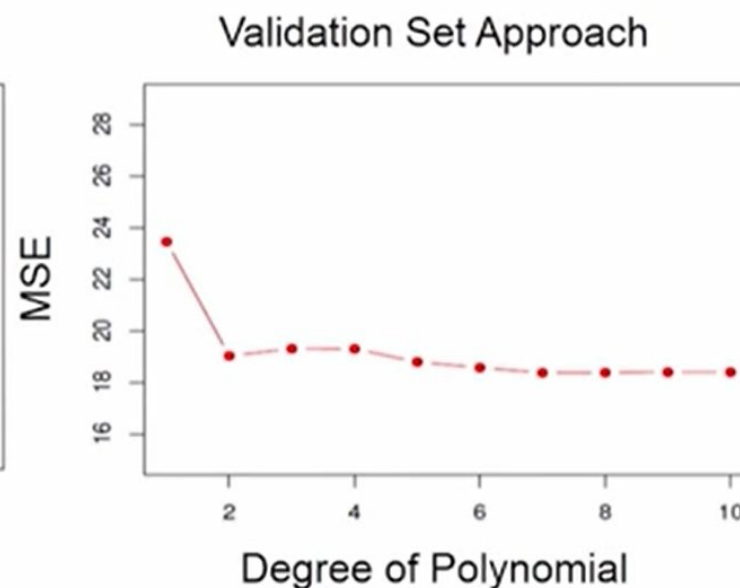
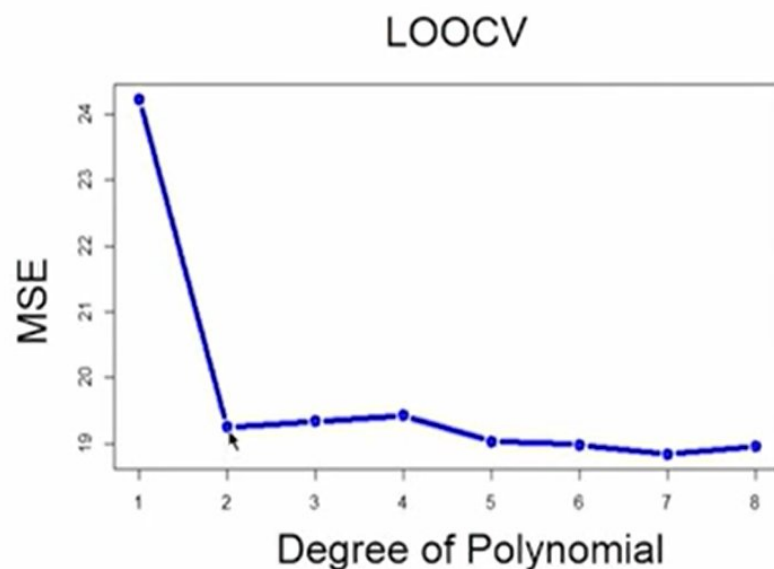
- Build model using $(n-1)$ samples and predict the response (y_i) for *the remaining sample*



$$CV_1 = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \hat{\boldsymbol{\beta}}^{(1)})^2$$

LOOCV: Example

- Example: $\text{mileage} \sim \text{horsepower}^1$
- Nonlinear Model: $\text{mileage} \sim f(\text{horsepower})$



¹Tibshirani et al (2013)

LOOCV

- Leave-one-out-cross-validation (LOOCV)
- Advantages
 - Far less bias comparison to the validation set approach
Training set contains $(n-1)$ observations each iteration
 - Yield the same results
No randomness in the training/validation set splits
 - Does not overestimate the test error rate as much as the validation set approach
- Disadvantages
 - Expensive to implement due to fitting happens n times
 - It may select a model of excessive size (more variables) than the optimal model

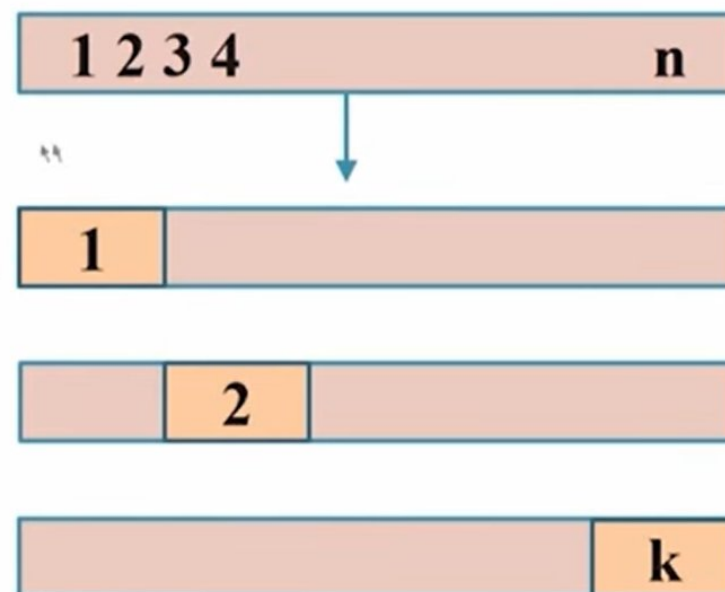
k-Fold Cross Validation

- Training data into k disjoint samples of equal size,

$$Z_1, Z_2, \dots, Z_k$$

- For each validation sample Z_i
 - Use remaining data to fit the model
 - Predict the response for the validation sample Z_i and compute mean square error (MSE_i),
 - Repeat for all k samples
 - The k -fold CV

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k MSE_i$$



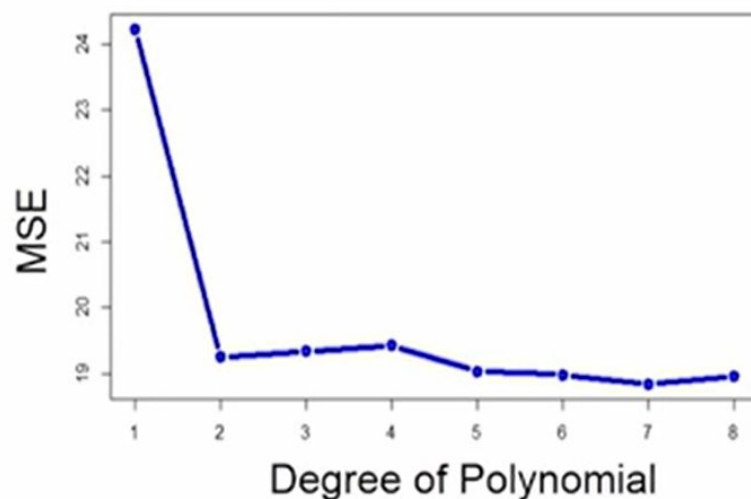
k-fold Validation

- For $k=n$, Leave-one-out-cross-validation (LOOCV)
- In practice, $k=5$ or 10 is taken,
- Less computation cost
- For computationally intensive learning methods
 - LOOCV fits the model n times
 - k -fold CV fits the model k times

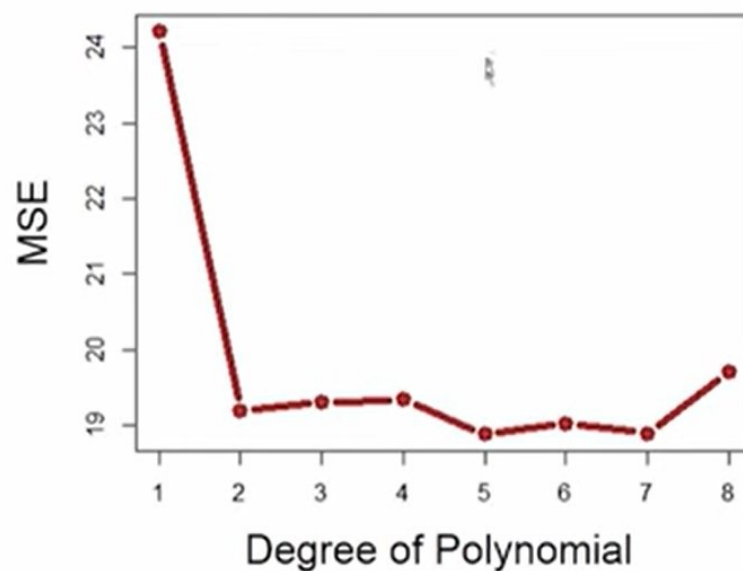
k-fold CV: Example

- Example: $\text{mileage} \sim \text{horsepower}^1$
- Nonlinear Model: $\text{mileage} \sim f(\text{horsepower})$

LOOCV



10-fold CV

¹Tibshirani et al (2013)

Multiple Linear Regression Model Building and Selection

Summary of simple linear regression

- Steps in building simple linear regression models
- Model summary
- Residual analysis
- Checking need for refinement
- Refined model building



In this lecture

- Multiple linear regression
 - Build linear model with one dependent and multiple independent variables
 - Look at summary of the model to discard insignificant variable
 - Model selection



Loading data

- Dataset 'nyc' is given in “.csv” format
- To load data from the file the function used is `read.csv( )`



read.csv()

Reads a file in table format and creates a data frame from it

SYNTAX

```
read.csv(file,row.names=1)
```

file	the name of the file which the data are to be read from. Each row of the table appears as one line of the file.
row.names	a vector of row names. This can be a vector giving the actual row names, or a single number giving the column of the table which contains the row names, or character string giving the name of the table column containing the row names.



Viewing data

- Assuming that 'nyc.csv' is in your current working directory

```
nyc <- read.csv("nyc.csv")
```

- The data is saved into a data frame 'nyc'



Viewing data

- `View(nyc)` will display the dataframe in a tabular format

	Price	Food	Decor	Service	East
1	43	22	18	20	0
2	32	20	19	19	0
3	34	21	13	18	0
4	41	20	20	17	0

- `head(nyc)` and `tail(nyc)` will display the first and last six rows from the dataframe



Description of dataset

Menu pricing in restaurants of NYC

y : Price of dinner

x_1 : Customer rating of the food (Food)

x_2 : Customer rating of the décor (Décor)

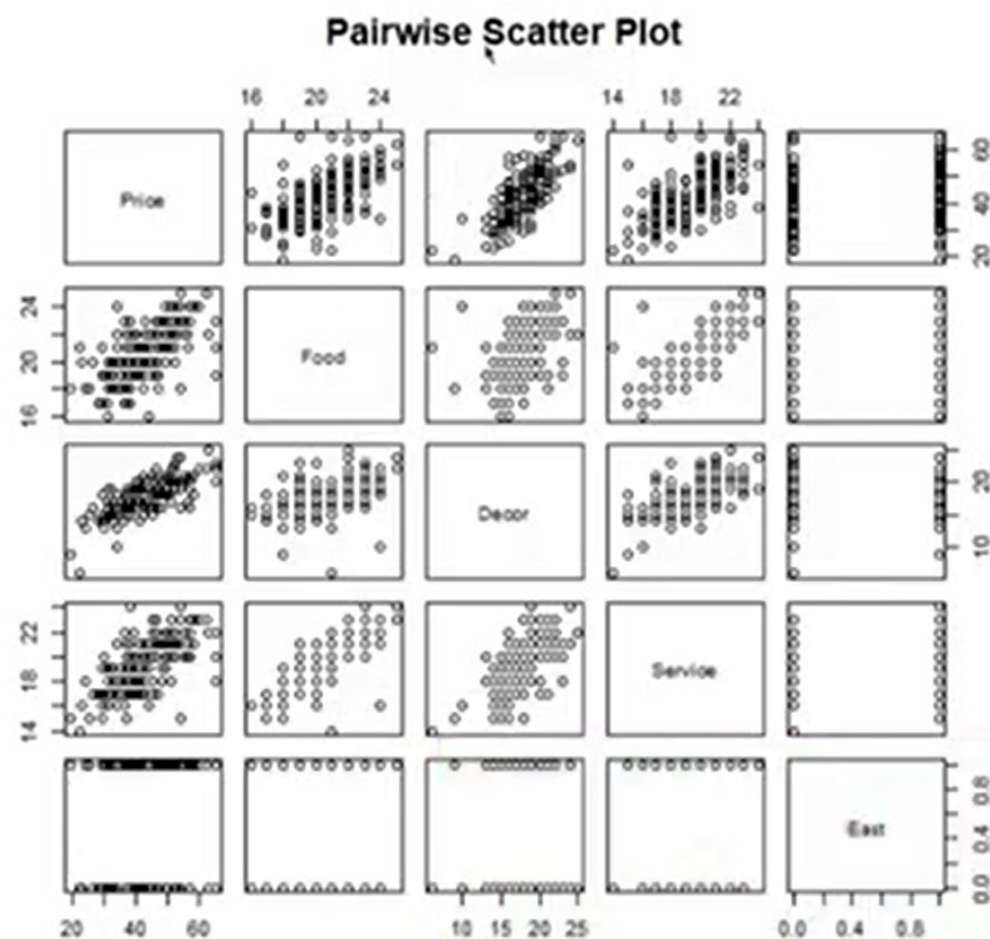
x_3 : Customer rating of the service (Service)

x_4 : If the restaurant is east or west (East)



Pairwise scatter plot

```
plot(nyc, main="Pairwise Scatter Plot")
```

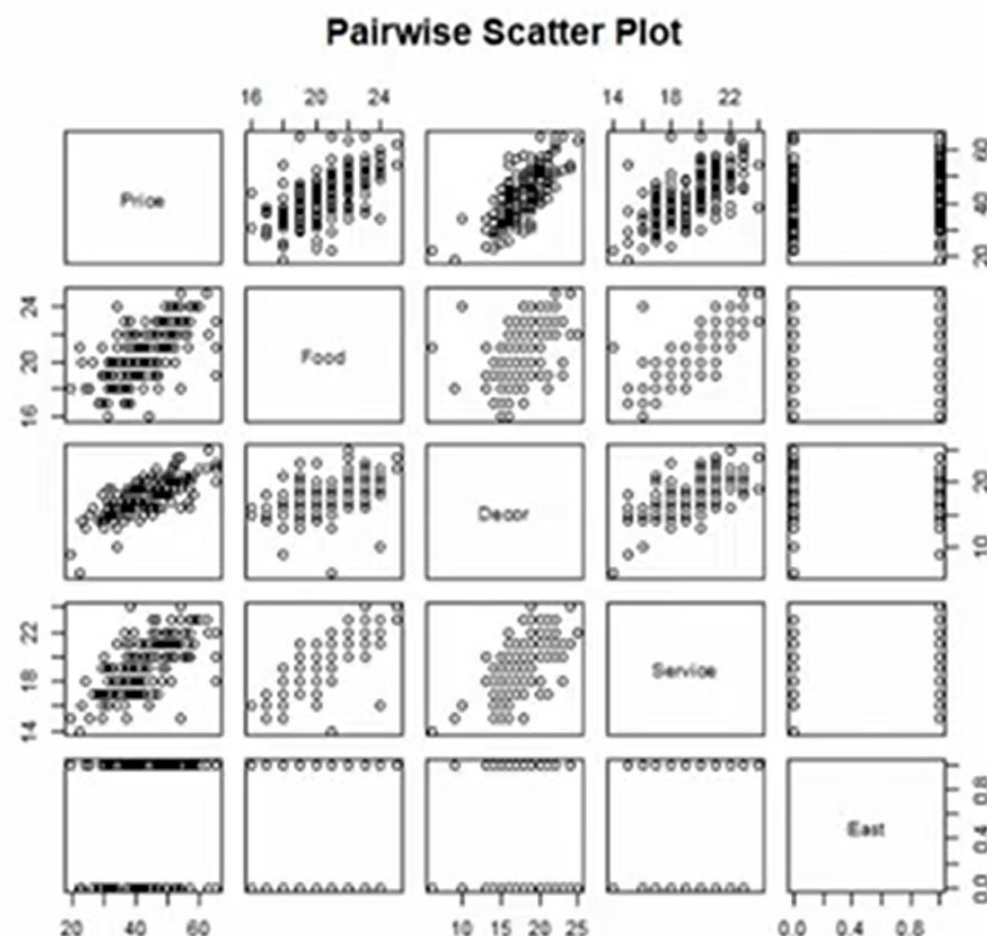


Pairwise scatter plot

```
plot(nyc, main="Pairwise Scatter Plot")
```

```
> round(corr(nyc),3)
```

	Price	Food	Decor	Service	East
Price	1.000	0.627	0.724	0.641	0.187
Food	0.627	1.000	0.504	0.795	0.180
Decor	0.724	0.504	1.000	0.645	0.036
Service	0.641	0.795	0.645	1.000	0.209
East	0.187	0.180	0.036	0.209	1.000



Model Building

Building multiple linear regression model

- Dependent variable (y) depends on p independent variables $x_i, i=1,2..p$

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p + \epsilon$$

- For i^{th} observation,

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{1,i} + \hat{\beta}_2 x_{2,i} + \cdots + \hat{\beta}_p x_{p,i} + \epsilon_i$$



Building multiple linear regression model

- Building multiple linear model using the function `lm()`

- Syntax: `lm(formula,data)`

`lm(dependent var~indep.var1+ indep.var2)`

```
nycmod_1 <- lm(Price~Food+Decor+Service+East,data = nyc)
```

or

```
nycmod_1<-lm(Price~.,data=nyc)
```



Model summary

```
nycmod_1 <- lm(Price~Food+Decor+Service+East,data = nyc)
```

```
summary(nycmod_1)
```

Call:

```
lm(formula = Price ~ Food + Decor + Service + East, data = nyc)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-14.0465	-3.8837	0.0373	3.3942	17.7491

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	-24.023800	4.708359	-5.102	9.24e-07	***
Food	1.538120	0.368951	4.169	4.96e-05	***
Decor	1.910087	0.217005	8.802	1.87e-15	***
Service	-0.002727	0.396232	-0.007	0.9945	
East	2.068050	0.946739	2.184	0.0304	*

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.738 on 163 degrees of freedom

Multiple R-squared: 0.6279, Adjusted R-squared: 0.6187

F-statistic: 68.76 on 4 and 163 DF, p-value: < 2.2e-16



New model dropping Service

```
nycmod_2 <- lm(Price~Food+Decor+East,data = nyc)
```

```
summary(nycmod_2)
```

Call:

```
lm(formula = Price ~ Food + Decor + East, data = nyc)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-14.0451	-3.8809	0.0389	3.3918	17.7557

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	-24.0269	4.6727	-5.142	7.67e-07	***
Food	1.5363	0.2632	5.838	2.76e-08	***
Decor	1.9094	0.1900	10.049	< 2e-16	***
East	2.0670	0.9318	2.218	0.0279	*

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.72 on 164 degrees of freedom

Multiple R-squared: 0.6279, Adjusted R-squared: 0.6211

F-statistic: 92.24 on 3 and 164 DF, p-value: < 2.2e-16

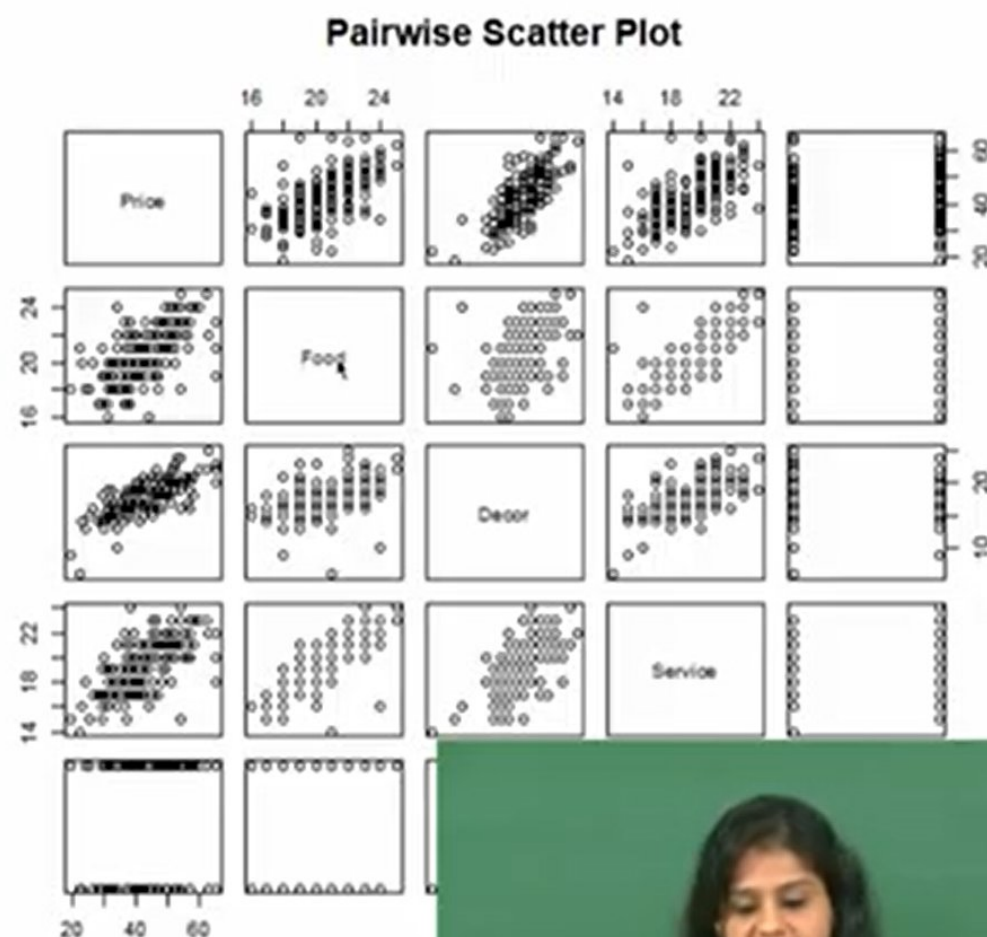


Pairwise scatter plot

```
plot(nyc, main="Pairwise Scatter Plot")
```

```
> round(cor(nyc),3)
```

	Price	Food	Decor	Service	East
Price	1.000	0.627	0.724	0.641	0.187
Food	0.627	1.000	0.504	0.795	0.180
Decor	0.724	0.504	1.000	0.645	0.036
Service	0.641	0.795	0.645	1.000	0.209
East	0.187	0.180	0.036	0.209	1.000



New model dropping Food

```
nycmod_3 <- lm(Price~Service+Decor+East,data = nyc)
```

```
summary(nycmod_3)
```

Call:

```
lm(formula = Price ~ Service + Decor + East, data = nyc)
```

Residuals:

Min	1Q	Median	3Q	Max
-14.514	-3.668	-0.769	3.704	18.778

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	-14.5308	4.3220	-3.362	0.000963	***
Service	1.1513	0.2973	3.872	0.000156	***
Decor	1.8956	0.2276	8.331	3.05e-14	***
East	2.1535	0.9927	2.169	0.031489	*

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.018 on 164 degrees of freedom

Multiple R-squared: 0.5882, Adjusted R-squared: 0.5807

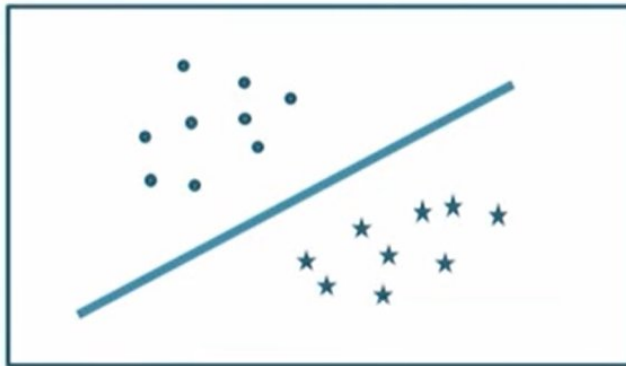
F-statistic: 78.09 on 3 and 164 DF, p-value: < 2.2e-16



Classification



Classification



- Classification – assigning classes to data points
 - Binary classification problems
 - Multi-class classification problems
- Linearly separable problems
- Linearly non-separable problems
 - Kernel methods



Logistic regression

Introduction

- Logistic regression is a classification technique
- Decision boundary (generally linear) derived based on probability interpretation
 - Results in a nonlinear optimization problem for parameter estimation
- Goal: Given a new data point, predict the class from which the data point is likely to have originated



Binary classification problem

- Classification is the task of identifying a category that a new observation belongs to based on the data with known categories
- When the number of categories is 2, it becomes a binary classification problem
- Binary classification is a simple “Yes” or “No” problem



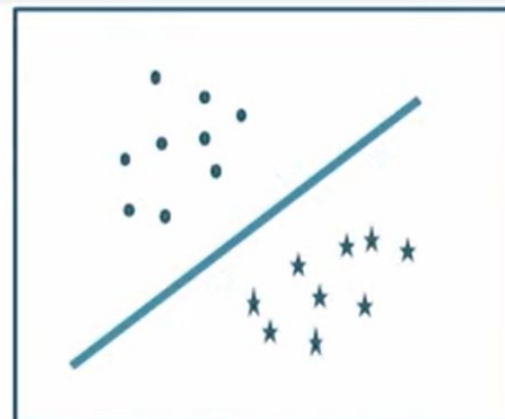
Input features

- Input features can be both qualitative and quantitative
- If the inputs are qualitative, then there has to be a systematic way of converting them to quantities
 - For example: A binary input like a “Yes” or “No” can be encoded as “1” and “0”
- Some data analytics approach can handle qualitative variables directly



Linear classifier

- Decision function is linear
- Binary classification can be performed depending on the side of the half-plane that the data falls in
- We saw this before in the linear algebra module
- However, simply guessing “yes” or “no” is pretty crude
- Can we do something better using probabilities ?



Output

- Why model probabilities ?
 - The probability of a “Yes” or “No” gives a better understanding of the sample’s membership to a particular category
 - Estimating the binary outputs from the probabilities is straight forward through simple thresholding
 - How does one model this probability ?



Linear and log models

- Make $p(x)$ a linear function of x

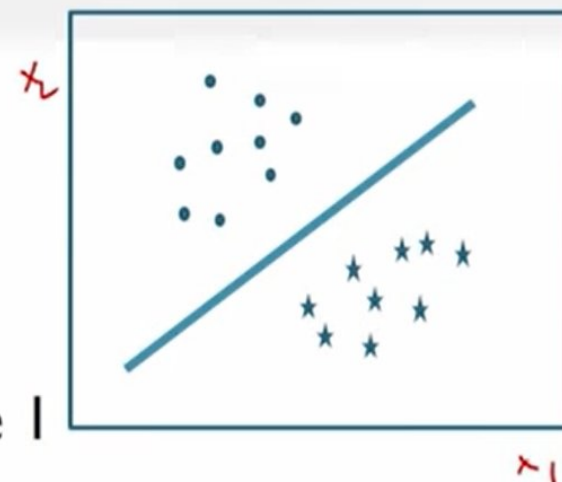
$$p(x) = \beta_0 + \beta_1 X$$

- This makes $p(x)$ unbounded below 0 and above 1
- Might give nonsensical results making it difficult to interpret them as probabilities

- Make $\log(p(x))$ a linear function of x

$$\log(p(x)) = \beta_0 + \beta_1 X$$

- Bounded only on one side

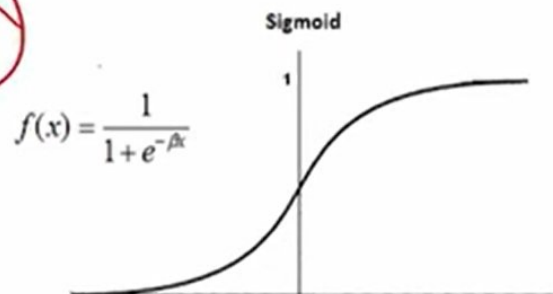


Sigmoid function

- Make $p(x)$ a sigmoid function of x

$$p(X) = \frac{e^{(\beta_0 + \beta_1 X)}}{1 + e^{(\beta_0 + \beta_1 X)}}$$

$$\text{or } \log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X$$



- $p(x)$ bounded above by 1 and below by 0
- Good modeling choice for real life scenarios
- The LHS can be interpreted as the log of odds-ratio in the second equation

Estimation of the parameters

- We find parameters in such a way that plugging these in the model equation should give the best possible classification for the inputs from both the classes
- This can be formalized by maximizing the following likelihood function

$$L(\beta_0, \beta_1) = \prod_{i=1}^n (p(x_i))^{y_i} (1 - p(x_i))^{(1-y_i)}$$

when x_i belongs to class 0, $y_i = 0$

when x_i belongs to class 1, $y_i = 1$

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2$$

Handwritten diagram showing a circle containing β_0 , β_1 , and β_2 , with an arrow pointing to the circle from the right.

$p(x)$



Log-likelihood function

- The log-likelihood function will become
$$l(\beta_0, \beta_1) = \sum_{i=1}^n y_i \log(p(x_i)) + (1 - y_i) \log(1 - p(x_i))$$
- Simplifying this expression and using the definition for $p(x)$ will result in an expression with the parameters of the linear decision boundary
- Now the parameters can be estimated by maximizing the above expression using any nonlinear optimization solver

x
 $\log x$



Logistic regression

Logit model

- The binary output for new samples can now be easily predicted using the following

$$p(x) = \frac{e^{(\beta_0 + \beta_1 X)}}{1 + e^{(\beta_0 + \beta_1 X)}}$$

- If $\beta_0 + \beta_1 X$ is non-negative then we get $p > 0.5$ and $Y=1$ otherwise we get $p < 0.5$ and $Y=0$
- Decision boundary is the equation $\beta_0 + \beta_1 X$



Example I

x_1	x_2
1	1
2	1
3	1
4	1
5	1
1	2
2	2
3	2
4	2
5	2

Class 0

x_1	x_2
6	3
7	3
8	3
9	3
10	3
6	4
7	4
8	4
9	4
10	4

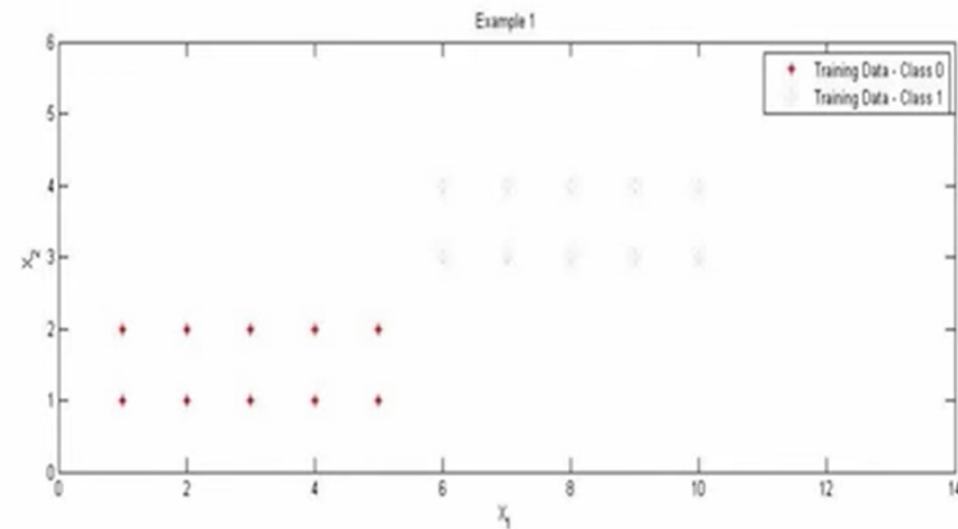
Class 1

x_1	x_2	Class?
1	3	?
2	3	?
4	4	?
5	4	?
3	3	?
6	2	?
9	2	?
8	1	?
7	2	?
10	1	?

Test Data



Example 1 continued



Results

- Input Features : X_1, X_2
- Classes : 0, 1
- Parameters:

$$\beta_0 = -42.5487$$

$$\beta_{11} = 2.9509$$

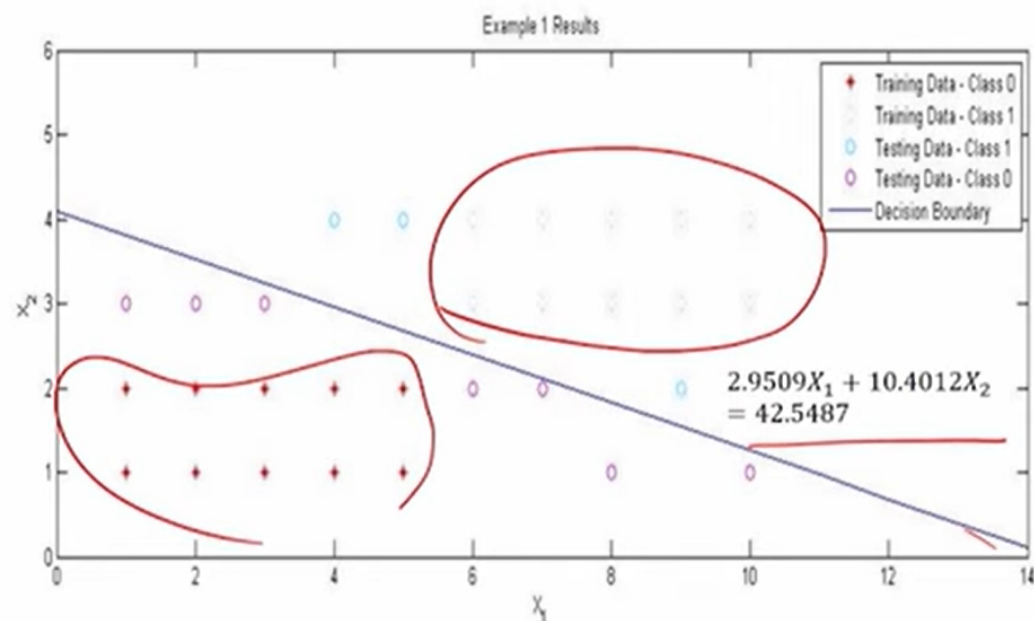
$$\beta_{12} = 10.4012$$

Test Results

X_1	X_2	Prob	Class
1	3	0.0002	0
2	3	0.004	0
4	4	0.999	1
5	4	0.999	1
3	3	0.076	0
6	2	0.0172	0
9	2	0.991	1
8	1	0.0002	0
7	2	0.251	0
10	1	0.0667	0



Example 1 solution



Regularization

- General objective
 - $\min_{\theta} -L(\theta)$
 - where $L(\theta) = (\sum_{i=1}^n y_i \log(p(x_i)) + (1 - y_i) \log(1 - p(x_i)))$
- When large number of independent variables are present, logistic regression tends to over-fit
- To prevent over-fitting, we need to penalize the coefficients
- This is known as regularization



Regularization

- Regularization helps in building non-complex models that avoids capturing noise in model due to over-fitting
- The objective now becomes
 - $\min_{\theta} -L(\theta) + \lambda * h(\theta)$ where λ is regularization parameter and $h(\theta)$ is regularization function
- Depending on $h(\theta)$, the regularization can be classified as L_1 or L_2 type
- $h(\theta) = \theta^T \theta$ for L_2 type regularization
- Larger the value of λ , more is the regularization strength
- Regularization helps the model work better on test data due to the fact that over-fitting is minimized on training data



Performance measures

Results : confusion matrix

```
# confusion matrix.
```

```
> confusionMatrix(crashTest_1_TEST$LogisPred,crashTest_1_TEST$CarType)
```

Confusion Matrix and Statistics

	Reference	
Prediction	Hatchback	SUV
Hatchback	10	1
SUV	0	9

Accuracy : 0.95

95% CI : (0.7513, 0.9987)

No Information Rate : 0.5

P-value [Acc > NIR] : 2.003e-05

Kappa : 0.9

McNemar's Test P-value : 1

Sensitivity : 1.0000

Specificity : 0.9000

Pos Pred Value : 0.9091

Neg Pred Value : 1.0000

Prevalence : 0.5000

Detection Rate : 0.5000

Detection Prevalence : 0.5500

Balanced Accuracy : 0.9500

'Positive' Class : Hatchback



Confusion matrix

		True condition	
Total population		Condition positive	Condition negative
Predicted condition	Predicted condition positive	True positive, Power	False positive, Type I error
	Predicted condition negative	False negative, Type II error	True negative

Source: https://en.wikipedia.org/wiki/Receiver_operating_characteristic



Measures of performance

- Terminology

- TP → true positives, TN → true negatives,
 - FP → false positives, FN → false negatives
- $$N = TP + TN + FP + FN$$
- TP – Correct identification of positive labels
 - TN – Correct identification of negative labels
 - FP – Incorrect identification of positive labels
 - FN – Incorrect identification of negative labels



Measures of performance

- Accuracy: Overall effectiveness of a classifier
 - $A = \frac{TP+TN}{N}$
 - Maximum value that accuracy can take is 1
 - This happens when the classifier exactly classifies two groups (i.e, $FP = 0$ and $FN = 0$)
- Remember
 - Total number of true positive labels = $TP+FN$
- Similarly
 - Total number of true negative labels = $TN+FP$



Measures of performance

- Sensitivity: Effectiveness of a classifier to identify positive labels
 - $S_e = \frac{TP}{TP + FN}$
- Specificity: Effectiveness of a classifier to identify negative labels
 - $S_p = \frac{TN}{FP + TN}$
- Both S_e and S_p lie between 0 *and* 1, 1 is an ideal value for each of them
- Balanced accuracy
 - $BA = (sensitivity + specificity)/2$



Measures of performance

- Prevalence: How often does the yes condition actually occur in our sample

$$P = \frac{TP + FN}{N}$$

- Positive predictive value: Proportion of correct results in labels identified as positive

$$PPV = \frac{(sensitivity * prevalence)}{((sensitivity * prevalence) + ((1 - specificity) * (1 - prevalence)))}$$

- Negative prediction value: Proportion of correct results in labels identified as negative

$$NPV = \frac{specificity * (1 - prevalence)}{(((1 - sensitivity) * prevalence) + ((specificity) * (1 - prevalence)))}$$

Measures of performance

- Detection rate:

- $DR = \frac{TP}{N}$

- Detection prevalence: prevalence of predicted events

- $DP = \frac{TP+FP}{N}$

- The Kappa statistic (or value) is a metric that compares an **observed accuracy** with an **expected accuracy** (random chance)

- $$\text{Kappa} = \frac{\text{observed accuracy} - \text{expected accuracy}}{1 - \text{expected accuracy}}$$



Measures of performance

- Observed accuracy

- $OA = \frac{a+d}{N}$

- Expected accuracy

- $EA = \frac{(a+c)(a+b) + (b+d)(c+d)}{N}$

- Kappa =
$$\frac{\frac{(a+d)}{N} - \left(\frac{(a+c)(a+b) + (b+d)(c+d)}{N} \right)}{\left(1 - \left(\frac{(a+c)(a+b) + (b+d)(c+d)}{N} \right) \right)}$$

- Where a, b, c and d are TP, FP, FN and TN respectively



Results : confusion matrix

confusion matrix.

> confusionMatrix(crashTest_1_TEST\$LogisPred,crashTest_1_TEST\$CarType)

Confusion Matrix and Statistics

	Reference	
Prediction	Hatchback	SUV
Hatchback	10	1
SUV	0	9

Accuracy : 0.95

95% CI : (0.7513, 0.9987)

No Information Rate : 0.5

P-Value [Acc > NIR] : 2.003e-05

Kappa : 0.9

McNemar's Test P-Value : 1

Sensitivity : 1.0000

Specificity : 0.9000

Pos Pred Value : 0.9091

Neg Pred Value : 1.0000

Prevalence : 0.5000

Detection Rate : 0.5000

Detection Prevalence : 0.5500

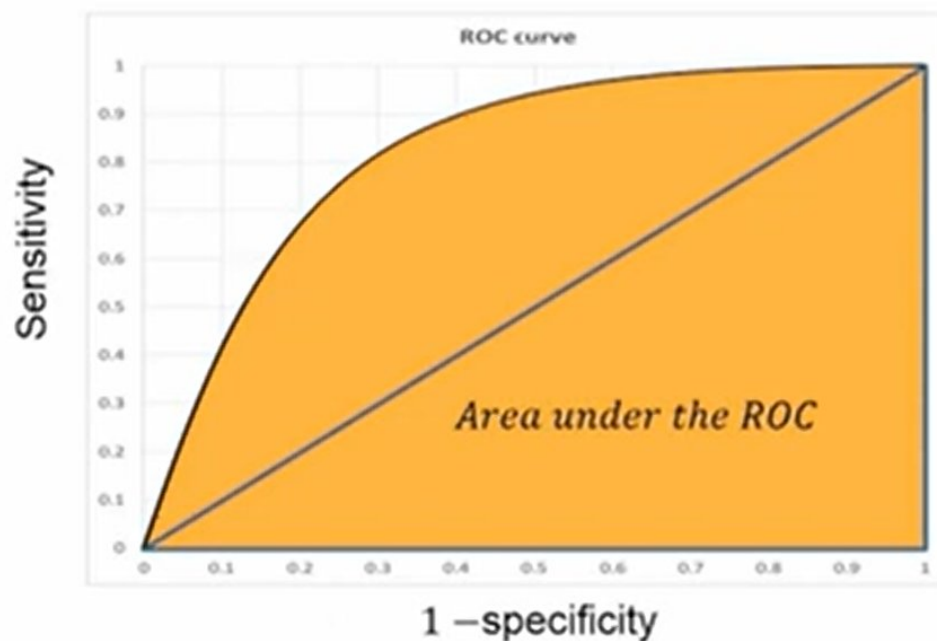
Balanced Accuracy : 0.9500

'Positive' Class : Hatchback



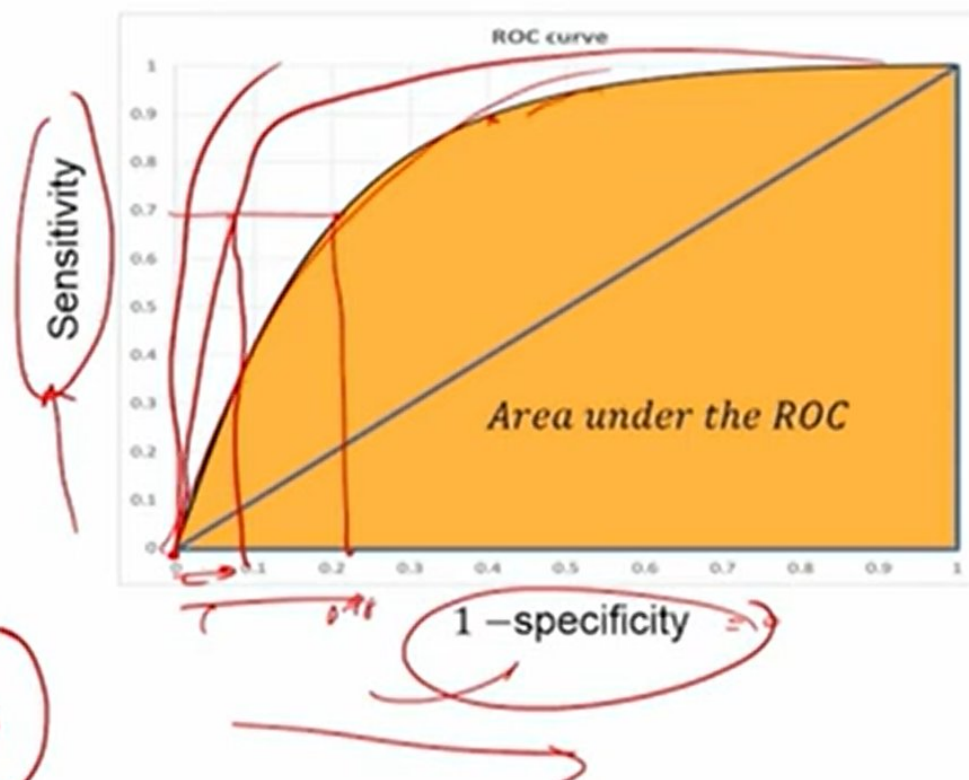
ROC

- ROC –An acronym for Receiver Operating Characteristics
- Originally developed and used in signal detection theory
- ROC graph:
 - Sensitivity as a function of specificity
 - sensitivity (Y-axis) and $1 - \text{specificity}$ (X-axis)



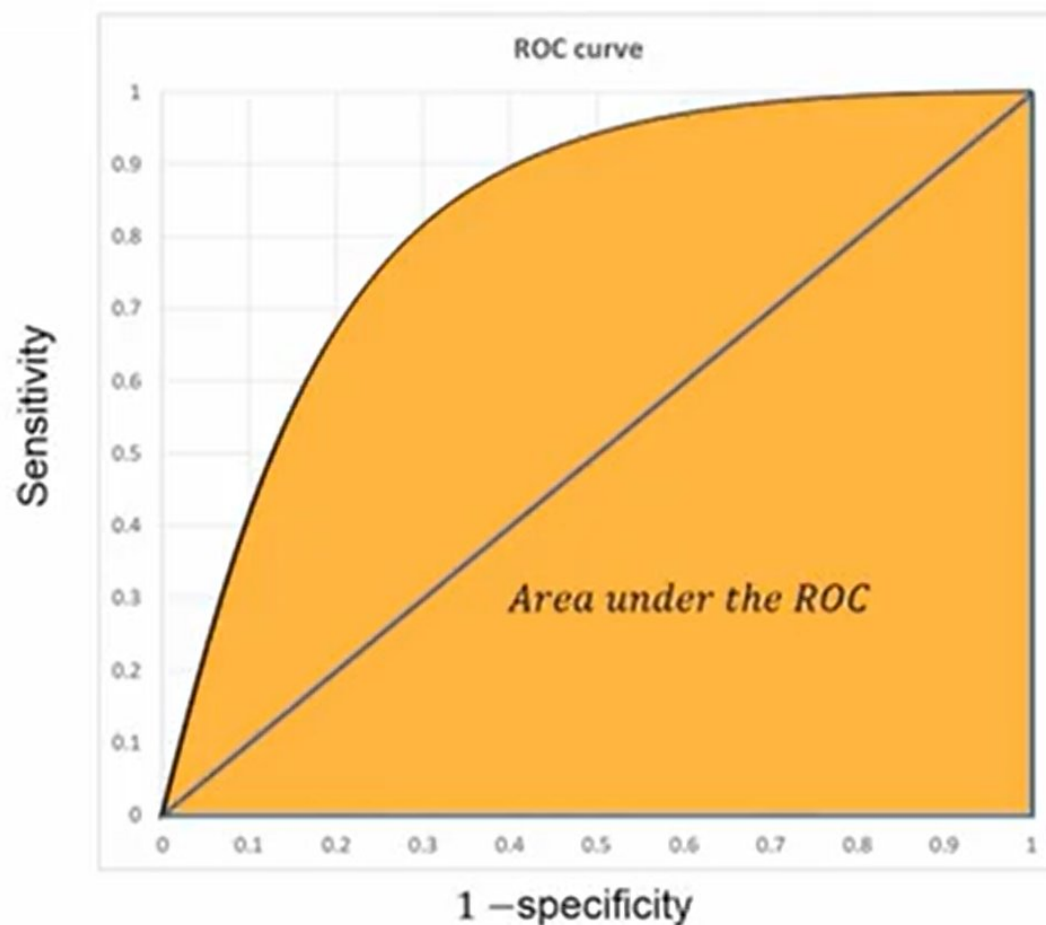
ROC

- ROC –An acronym for Receiver Operating Characteristics
- Originally developed and used in signal detection theory
- ROC graph:
 - Sensitivity as a function of specificity
 - sensitivity (Y-axis) and 1 –specificity (X-axis)



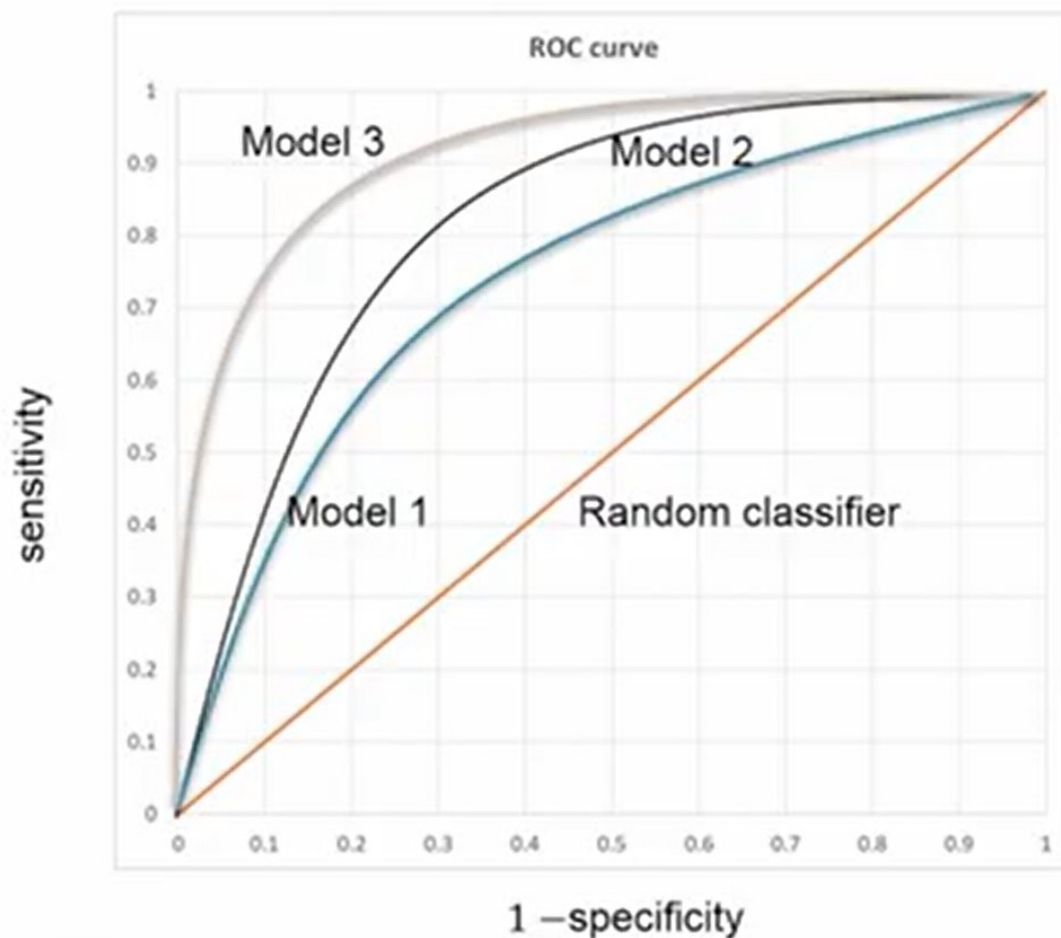
ROC

- ROC can be used to
 - See the classifier performance at different threshold levels (from 0 to 1)
 - AUC- Area under the ROC
 - An area of 1 represents a perfect test; an area of 0.5 represents a worthless model.
 - .90 – 1 = excellent
 - .80 – .90 = good
 - .70 – .80 = fair
 - .60 – .70 = poor
 - $AUC < 0.5$, check whether your labels are marked in opposite



ROC

- ROC can be used to
 - Compare different classifiers at one threshold or overall threshold levels
 - Performance
 - Model 3 > Model 2 > Model 1



Logistic Regression implementation in R

In this lecture

- Case study
 - Problem statement
- Solve the case study using R
 - Read the data from a “.csv” file
 - Understand the data
 - glm() function
 - Interpret the results

Key points from previous lecture

- Logistic Regression is primarily used as a classification algorithm
- It is supervised learning algorithm
 - Data is labelled
- Parametric approach
- Decision boundary derived based on probability interpretation
- Decision boundary can be linear/ non-linear



Automotive Crash Testing

Automotive Crash Testing- Problem Statement

- A crash test is a form of destructive testing that is performed in order to ensure high safety standards for various cars



Automotive Crash Testing



Hatchback



SUV

Automotive Crash Testing- Problem Statement

- Several cars have rolled into an independent audit unit for crash test
- They are being evaluated on a defined scale {poor (-10) to excellent(10)} on:
 - 1) Manikin head impact
 - 2) Manikin body impact
 - 3) Interior impact
 - 4) HVAC impact
 - 5) Safety alarm system



Automotive Crash Testing

- Each crash test is very expensive
- The crash test was performed for only 100 cars
- Type of car- Hatchback/SUV, was noted
- However with this data in future they should be able to predict the type of the car
- Part of data reserved for building a model and remaining kept for analysis



Automotive Crash Testing

- Data for 80 cars is given in crashTest_1.csv
- Data for remaining 20 cars is given in crashTest_1_TEST.csv
- Use **logistic regression** classification technique to classify the car types as Hatchback/SUV



Solution to case study using R



Getting things ready

- Setting working directory, clearing variables in the workspace
- Installing or loading required packages

```
# Set the working directory as the directory which  
# contains the data files  
# setwd("Path of the directory with data files")  
rm(list=ls()) # to clear the environment  
# install.packages("caret", dependencies = TRUE)  
  
library(caret) # for confusionMatrix
```



Reading the data

- Data for this case study is provided to you in files with names
 - crashTest_1- training data
 - crashTest_1_TEST- testing data
- To read the data from a “.csv” file we use `read.csv()` function



Reading the data

Reads a file in table format and creates a data frame from it

SYNTAX

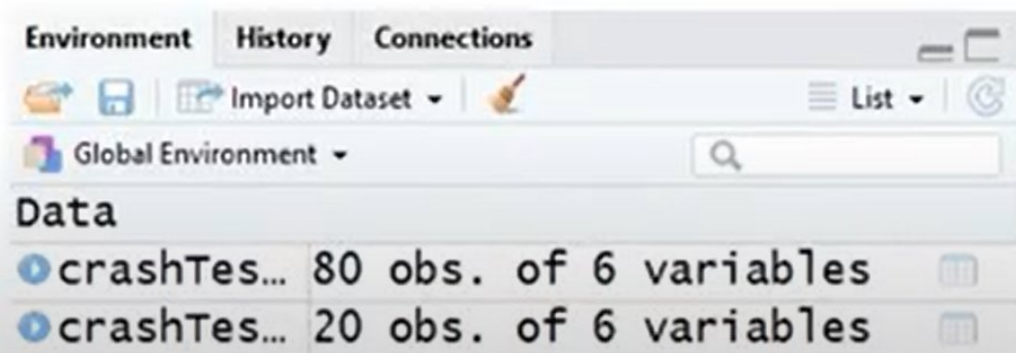
```
read.csv(file,row.names=1)
```

file	the name of the file which the data are to be read from. Each row of the table appears as one line of the file.
row.names	a vector of row names. This can be a vector giving the actual row names, or a single number giving the column of the table which contains the row names, or character string giving the name of the table column containing the row names.



Reading the data

```
#Reading the data  
crashTest_1<-read.csv("crashTest_1.csv",row.names=1)  
crashTest_1_TEST<-read.csv("crashTest_1_TEST.csv",row.names=1)
```



The screenshot shows the RStudio 'Environment' pane. At the top, there are tabs for 'Environment', 'History', and 'Connections'. Below the tabs are icons for file operations and a search bar. The 'Global Environment' is selected, and a search bar is visible. Under the 'Data' section, two objects are listed: 'crashTes...' with 80 observations and 6 variables, and another 'crashTes...' with 20 observations and 6 variables. Each entry has a small icon to its right.

Data		
crashTes...	80 obs. of 6 variables	
crashTes...	20 obs. of 6 variables	

Viewing the data

```
View(crashTest_1)
```



	ManHI	ManBI	Intl	HVAGI	Safety	CarType
1	-5.27	-1.30	2.86	-4.85	4.04	SUV
2	-4.82	-5.38	9.72	-0.97	-4.57	Hatchback



Understanding the data

- crashTest_1 contains 80 observations of 6 variables
- crashTest_1_TEST contains 20 observations of 6 variables
- The variables are: Manikin head impact, Manikin body impact, Interior impact, HVAC impact, Safety alarm system



Understanding the data

- `crashTest_1` contains 80 observations of 6 variables
- `crashTest_1_TEST` contains 20 observations of 6 variables
- The variables are: Manikin head impact, Manikin body impact, Interior impact, HVAC impact, Safety alarm system
 - First five columns are the details about the car and last column is the label which says whether the car type Hatchback/ SUV



Structure of the data

- Structure of data
 - Variables and their data types
- `str()`

SYNTAX

```
str(object)
```

object	any R object about which you want to have some information.
--------	---



Structure of train data

```
> str(crashTest_1)
'data.frame': 80 obs. of 6 variables:
 $ ManHI : num -5.27 -4.82 9.57 2.84 0 0.4 5.94 5.78 0.86 7.36 ...
 $ ManBI : num -1.3 -5.38 -7.5 -2.85 2.68 6.34 3.14 -1.75 -4.32 7.42 ...
 $ IntI : num 2.86 9.72 -7.61 0.92 -4.15 0.83 -6.65 -6.85 8.1 0.27 ...
 $ HVACi : num -4.85 -0.97 1.33 5.51 0.85 5.03 6.62 0.73 -8.96 -8.62 ...
 $ Safety : num 4.04 -4.57 -5.1 -6.64 5.58 -8.1 -1.32 5.5 3.1 3.08 ...
 $ CarType: Factor w/ 2 levels "Hatchback","SUV": 2 1 1 1 2 2 1 1 1 2 ...
```



Summary of the data

- Summary of data
 - The function invokes particular methods which depend on the class of the first argument.
- `summary()`

Summary gives a 5`point summary for numeric attributes in the data

SYNTAX

```
summary(object)
```



Summary of crashTest_1

```
> summary(crashTest_1)
```

ManHI		ManBI	
Min.	:-9.9300	Min.	:-9.9400
1st Qu.	:-5.1950	1st Qu.	:-5.7050
Median	: 0.6350	Median	:-1.8150
Mean	:-0.0935	Mean	:-0.9277
3rd Qu.	: 5.0500	3rd Qu.	: 3.4175
Max.	: 9.5700	Max.	: 9.6100

IntI		HVACi	
Min.	:-9.9900	Min.	:-9.8200
1st Qu.	:-5.5725	1st Qu.	:-5.6750
Median	:-0.4150	Median	: 0.8700
Mean	:-0.1349	Mean	: 0.1197
3rd Qu.	: 4.9775	3rd Qu.	: 5.0625
Max.	: 9.7200	Max.	: 9.8900

Safety		CarType	
Min.	:-9.8000	Hatchback	:50
1st Qu.	:-4.6775	SUV	:30
Median	: 0.8300		
Mean	: 0.5437		
3rd Qu.	: 4.6225		
Max.	: 9.9900		



Summary of crashTest_1_TEST

```
> summary(crashTest_1_TEST)
```

ManHI		ManBI	
Min.	:-9.940	Min.	:-8.740
1st Qu.:	-5.535	1st Qu.:	-2.502
Median :	0.740	Median :	0.670
Mean :	0.047	Mean :	0.328
3rd Qu.:	5.110	3rd Qu.:	2.500
Max.	: 9.090	Max.	: 8.420

IntI		HVACi	
Min.	:-8.950	Min.	:-9.2300
1st Qu.:	-3.272	1st Qu.:	-2.4550
Median :	1.200	Median :	0.6750
Mean :	0.524	Mean :	0.7235
3rd Qu.:	3.908	3rd Qu.:	5.3375
Max.	: 8.870	Max.	: 8.3300

Safety		CarType	
Min.	:-8.660	Hatchback:	10
1st Qu.:	-6.095	SUV	:10
Median :	-0.770		
Mean :	0.191		
3rd Qu.:	4.992		
Max.	: 9.620		



glm()

```
glm(formula, data, family)
```

Arguments

formula	object of class "formula" (or one that can be coerced to that class): a symbolic description of the model to be fitted
data	dataframe containing variables
family	a description of the error distribution and link function to be used in the model. For glm this can be a character string naming a family function, a family function or the result of a call to a family function In specific, family='binomial' corresponds to logistic regression



Building a logistic regression model

```
# Model
logisfit<-glm(formula = crashTest_1$CarType~., family = 'binomial',
              data = crashTest_1)
```

$$p(X) = \frac{e^{(\beta_0 + \beta_1 X)}}{1 + e^{(\beta_0 + \beta_1 X)}}$$

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X$$

```
> logisfit
```

```
Call: glm(formula = crashTest_1$CarType ~ ., family = "binomial", data = crashTest_1)
```

```
Coefficients:
```

(Intercept)	ManHI	ManBI	IntI	HVACi	Safety
-22.76	-13.48	36.02	-44.90	-58.50	-27.36

```
Degrees of Freedom: 79 Total (i.e. Null); 74 Residual
```

```
Null Deviance: 105.9
```

```
Residual Deviance: 5.359e-08 AIC: 12
```

Summary of model

```
> summary(logisfit)
```

Call:

```
glm(formula = crashTest_1$CarType ~ ., family = "binomial", data = crashTest_1)
```

Deviance Residuals:

	Min	1Q	Median	3Q	Max
	-1.316e-04	-2.100e-08	-2.100e-08	2.100e-08	1.266e-04

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-22.76	12007.54	-0.002	0.998
ManHI	-13.48	3077.29	-0.004	0.997
ManBI	36.02	7221.18	0.005	0.996
IntI	-44.90	8853.08	-0.005	0.996
HVACi	-58.50	11461.92	-0.005	0.996
Safety	-27.36	5396.42	-0.005	0.996

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1.0585e+02 on 79 degrees of freedom
 Residual deviance: 5.3590e-08 on 74 degrees of freedom
 AIC: 12

Number of Fisher Scoring iterations: 25

Finding the odds

- `predict()`
- Syntax: `predict(object)`

Finding the odds

```
logisTrain<-predict(logisfit, type = 'response')
```

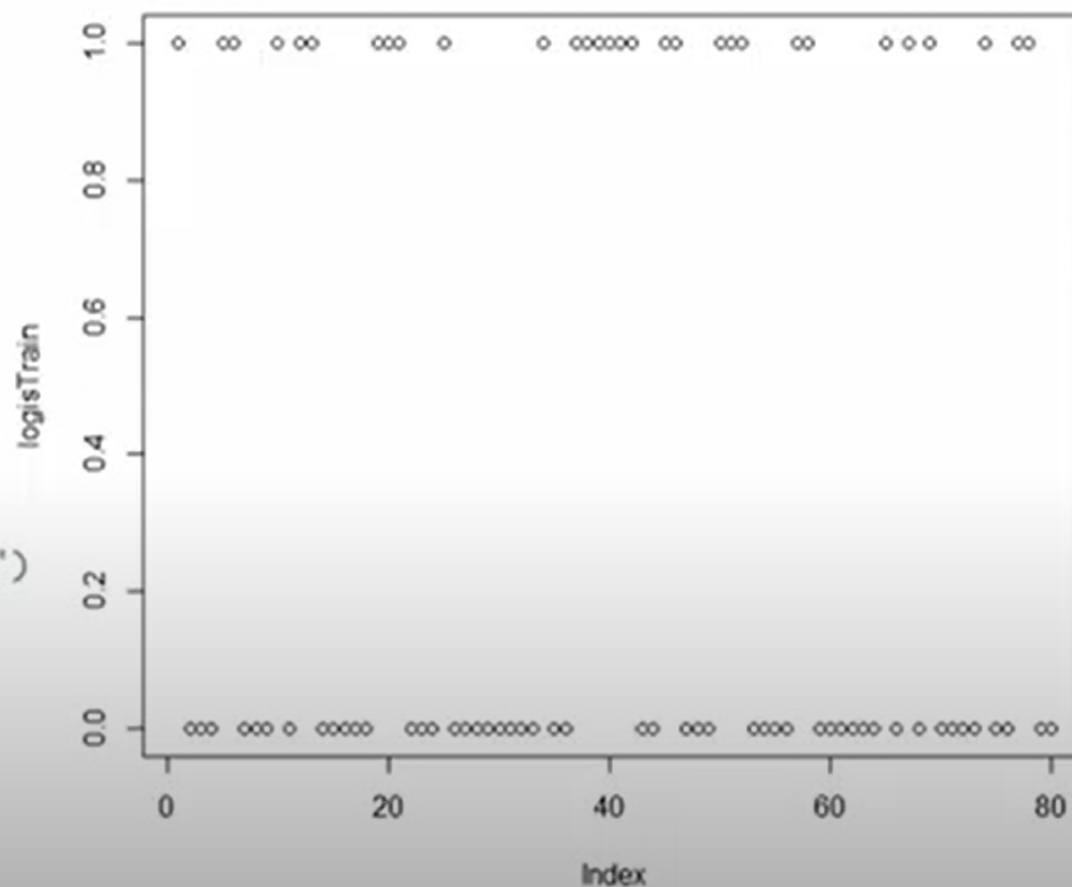
- `predict()` with `type='response'` gives probabilities
- By default otherwise it returns `log(odds)`



Plotting the probabilities

```
plot(logisTrain)
```

```
# Finding the odds  
logisTrain<-predict(logisfit, type = 'response')
```



Identifying probabilities associated with the CarType

- Mean of probabilities
- This helps us identify the probabilities associated with the two classes

```
> tapply(logisTrain,crashTest_1$CarType,mean)
      Hatchback          SUV 
2.851316e-10 1.000000e+00
```



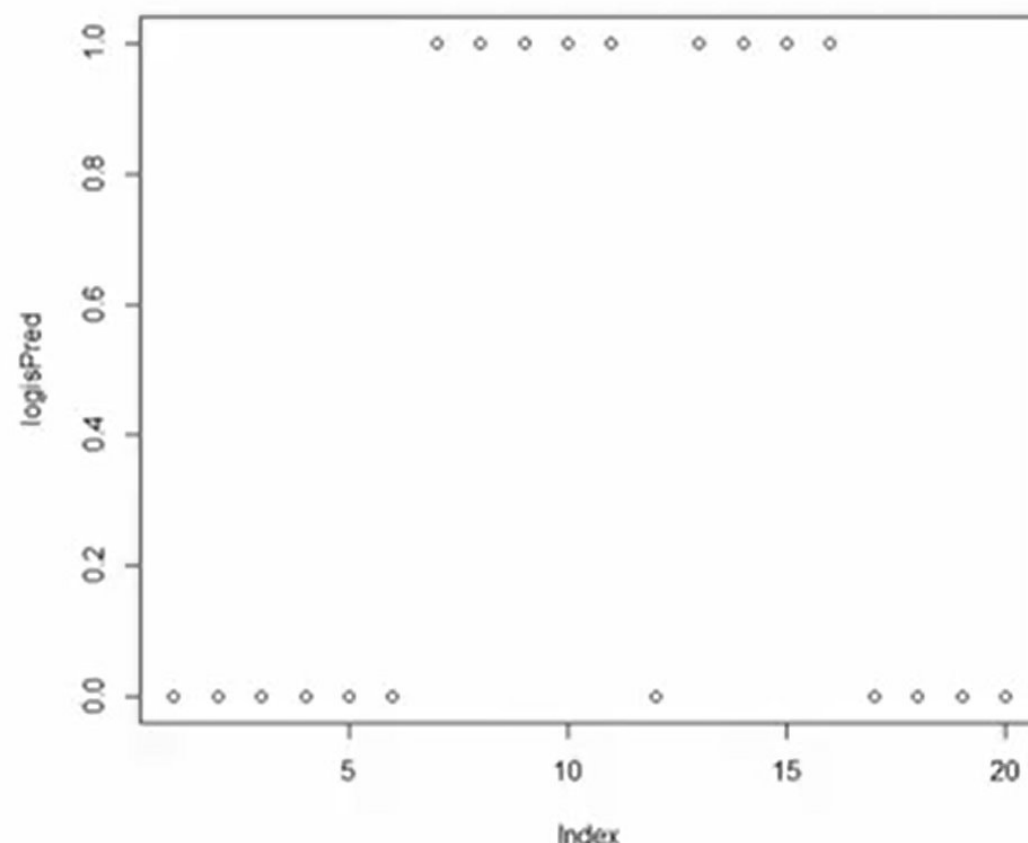
Predicting on test data

```
# Predicting on test set
logisPred<-predict(logisfit,
                  newdata = crashTest_1_TEST,
                  type='response')

plot(logisPred)
```

- “logisPred” is the output which has the probability values

```
> logisPred
      81      82      83      84
4.269083e-13 1.740691e-04 2.220446e-16 2.220446e-16
```



Results

- We classify whether the test point is Hatchback/ SUV by setting a threshold

```
crashTest_1_TEST[logisPred<=0.5,"LogisPred"]<-"Hatchback"  
crashTest_1_TEST[logisPred>0.5,"LogisPred"]<-"SUV"
```



The screenshot shows the RStudio interface with three tabs: 'crashTest_1_TEST', 'Classification+Clustering R session cod...', and 'lda_logistic_final.R'. The 'crashTest_1_TEST' tab is active, displaying a data table with 8 columns: 'ManHI', 'ManBI', 'IntI', 'HVACI', 'Safety', 'CarType', and 'LogisPred'. The table contains two rows of data, indexed 81 and 82. Row 81 has values 1.94, 2.21, 3.38, 1.78, -7.19, 'Hatchback', and 'Hatchback'. Row 82 has values -0.02, -3.33, 0.79, -6.63, 7.99, 'SUV', and 'Hatchback'.

	ManHI	ManBI	IntI	HVACI	Safety	CarType	LogisPred
81	1.94	2.21	3.38	1.78	-7.19	Hatchback	Hatchback
82	-0.02	-3.33	0.79	-6.63	7.99	SUV	Hatchback



Confusion matrix

```
confusionMatrix(table(crashTest_1_TEST[,7],
                      crashTest_1_TEST[,6]),positive = 'Hatchback')
```

Confusion Matrix and Statistics

Prediction	Reference	
	Hatchback	SUV
Hatchback	10	1
SUV	0	9

Accuracy : 0.95
 95% CI : (0.7513, 0.9987)
 No Information Rate : 0.5
 P-Value [Acc > NIR] : 2.003e-05

 Kappa : 0.9
 Mcnemar's Test P-Value : 1

Confusion matrix

```
confusionMatrix(table(crashTest_1_TEST[,7],  
                      crashTest_1_TEST[,6]),positive = 'Hatchback')
```

```
Sensitivity : 1.0000  
Specificity : 0.9000  
Pos Pred Value : 0.9091  
Neg Pred Value : 1.0000  
Prevalence : 0.5000  
Detection Rate : 0.5000  
Detection Prevalence : 0.5500  
Balanced Accuracy : 0.9500  
  
'Positive' Class : Hatchback
```